

20 Production & Interview-Level Problems Solved Strictly via Nested Subqueries, WHERE, GROUP BY, and HAVING

PART 1: ADVANCED FILTERING & SUBQUERY PREDICATES

Problem 01

INTERVIEW TIER

Find the registration entries from the **Students** table for individuals whose roll number is strictly greater than the overall average roll number within the dataset.

Tests: Scalar subquery evaluation inside a numeric conditional clause.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE roll > (
    SELECT AVG(roll)
    FROM Students
)
ORDER BY roll ASC;
```

Problem 02

INTERVIEW TIER

Isolate the IDs of students who have scored 78 marks or lower in more than one subject execution loop.

Tests: Intersecting multi-row evaluation constraints via sub-grouped aggregation structures.

SQL QUERY SOLUTION:

```
SELECT std_id
FROM Scores
WHERE marks <= 78
GROUP BY std_id
HAVING COUNT(sub_id) > 1;
```

Problem 03 ELITE TIER

Find all student records whose total accumulated marks across all subjects exceed the combined global average score of all active entries in the database.

Tests: Grouping comparison models where aggregate targets are checked against dynamic scalar values.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE id IN (
    SELECT std_id
    FROM Scores
    GROUP BY std_id
    HAVING SUM(marks) > (SELECT AVG(marks) FROM Scores)
);
```

Problem 04 INTERVIEW TIER

Identify the subject ID containing the single highest score standard variance, defined as the mathematical difference between its top score and bottom score.

Tests: Combining computed expressions with grouping limits and absolute sort orders.

SQL QUERY SOLUTION:

```
SELECT sub_id
FROM Scores
GROUP BY sub_id
ORDER BY (MAX(marks) - MIN(marks)) DESC
LIMIT 1;
```

Problem 05 INTERVIEW TIER

Find the details of students who have completed exams in every single subject registered in the system's Subjects directory table.

Tests: Implementing full relational division logic utilizing aggregate count equivalence statements.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE id IN (
    SELECT std_id
    FROM Scores
    GROUP BY std_id
    HAVING COUNT(DISTINCT sub_id) = (SELECT COUNT(*) FROM Subjects)
);
```

Problem 06 ELITE TIER

Find the IDs of subjects that have received exactly the same number of evaluations or test entries as Subject ID 2 (Sanskrit). Do not include Subject ID 2 in the final response set.

Tests: Comparative target mapping across groups using nested scalar results.

SQL QUERY SOLUTION:

```
SELECT sub_id
FROM Scores
WHERE sub_id != 2
GROUP BY sub_id
HAVING COUNT(std_id) = (
    SELECT COUNT(std_id)
    FROM Scores
    WHERE sub_id = 2
);
```

Problem 07 INTERVIEW TIER

Identify the names of students belonging to classes starting with the numeric prefix '6' (e.g., '6A', '6B') who have logged any score below 70.

Tests: Row validation using textual wildcards combined with multi-column exclusion arrays.

SQL QUERY SOLUTION:

```
SELECT name FROM Students
WHERE class LIKE '6%'
AND id IN (
    SELECT std_id
    FROM Scores
    WHERE marks < 70
);
```

PART 2: ADVANCED GROUP-BY & AGGREGATE BENCHMARKS

Problem 08

INTERVIEW TIER

Find the ID of the subject that holds the lowest internal score average across all evaluations recorded in the system.

Tests: Identifying statistical extrema via custom sorting over mathematical groupings.

SQL QUERY SOLUTION:

```
SELECT sub_id
FROM Scores
GROUP BY sub_id
ORDER BY AVG(marks) ASC
LIMIT 1;
```

Problem 09

ELITE TIER

Find the IDs of all students who achieved a higher score in any subject than the specific maximum mark obtained in Subject ID 1 (Math).

Tests: Correlated logic checking individual scores against a strict table-wide maximum value.

SQL QUERY SOLUTION:

```
SELECT DISTINCT std_id
FROM Scores
WHERE marks > (
    SELECT MAX(marks)
    FROM Scores
    WHERE sub_id = 1
);
```

Problem 10 INTERVIEW TIER

Retrieve the master table records for the top 3 performing students based on their computed high-score performance values (maximum score achieved in any single subject).

Tests: Mapping data using composite filtering limits aligned with targeted subsets.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE id IN (
    SELECT std_id
    FROM Scores
    GROUP BY std_id
    ORDER BY MAX(marks) DESC
)
LIMIT 3;
```

Problem 11 ELITE TIER

Isolate all scores records that are exactly equal to the second highest mark recorded across the entire database table.

Tests: Emulating offset-based rankings via relative subquery exclusions.

SQL QUERY SOLUTION:

```
SELECT * FROM Scores
WHERE marks = (
    SELECT MAX(marks)
    FROM Scores
    WHERE marks < (SELECT MAX(marks) FROM Scores)
);
```

Problem 12 INTERVIEW TIER

Identify all student records where the student's primary internal ID matches perfectly with the ID of any subject registered in the directory.

Tests: Cross-entity structural mapping using scalar key matching rules.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE id IN (
    SELECT id
    FROM Subjects
);
```

Problem 13

INTERVIEW TIER

Find the complete data rows from the Scores table for records where the mark value falls within the precise mathematical range of 65 and 80 inclusive, excluding all data linked to Student ID 3.

Tests: Complex boolean combinations inside standard query filters.

SQL QUERY SOLUTION:

```
SELECT * FROM Scores
WHERE marks BETWEEN 65 AND 80
      AND std_id != 3
ORDER BY marks DESC;
```

Problem 14

ELITE TIER

Isolate the IDs of students who have shown inconsistent testing behavior, defined here as having an absolute gap of 15 or more marks between their personal highest and personal lowest exam performances.

Tests: Evaluation of range-based thresholds on grouped attributes using HAVING filters.

SQL QUERY SOLUTION:

```
SELECT std_id
FROM Scores
GROUP BY std_id
HAVING (MAX(marks) - MIN(marks)) >= 15;
```

Problem 15

INTERVIEW TIER

Count how many individual students have logged scores across more than two unique, non-overlapping subject instances.

Tests: Using subqueries as derived counts to evaluate cardinality benchmarks.

SQL QUERY SOLUTION:

```
SELECT COUNT(*) AS total_qualifying_students
FROM (
  SELECT std_id
  FROM Scores
  GROUP BY std_id
  HAVING COUNT(DISTINCT sub_id) > 2
) AS derived_student_set;
```

Problem 16 ELITE TIER

Find the IDs of subjects whose individual calculated subject-wide average mark is strictly lower than the overall global average mark calculated from every score entry in the database.

Tests: Group evaluation using nested statistical parameters.

SQL QUERY SOLUTION:

```
SELECT sub_id
FROM Scores
GROUP BY sub_id
HAVING AVG(marks) < (
    SELECT AVG(marks)
    FROM Scores
);
```

Problem 17 INTERVIEW TIER

Find the student records from the database who belong to any section of Class 6 or Class 8 (e.g., '6A', '8C') and have a roll number configured below 600.

Tests: Complex conditional scoping combining multiple text patterns with numeric bounds.

SQL QUERY SOLUTION:

```
SELECT * FROM Students
WHERE (class LIKE '6%' OR class LIKE '8%')
AND roll < 600
ORDER BY class ASC;
```

Problem 18 ELITE TIER

Identify the IDs of subjects that have never seen a single evaluation record score go above 80 marks.

Tests: Complex negative mapping using subqueries to find elements that don't match a condition.

SQL QUERY SOLUTION:

```
SELECT id FROM Subjects
WHERE id NOT IN (
    SELECT DISTINCT sub_id
    FROM Scores
    WHERE marks > 80
);
```

Problem 19**INTERVIEW TIER**

Find the IDs of all students who have logged exactly one score record inside the database table, ignoring all multi-test profiles.

Tests: Cardinality screening on unique entities using simple aggregate metrics.

SQL QUERY SOLUTION:

```
SELECT std_id
FROM Scores
GROUP BY std_id
HAVING COUNT(sub_id) = 1;
```

Problem 20**ELITE TIER**

Find the complete data rows from the Subjects table where the subject has at least 3 distinct evaluation scores entered into the database, ordered by the subject name descending.

Tests: Filtering reference values based on transactional frequencies.

SQL QUERY SOLUTION:

```
SELECT * FROM Subjects
WHERE id IN (
    SELECT sub_id
    FROM Scores
    GROUP BY sub_id
    HAVING COUNT(marks) >= 3
)
ORDER BY name DESC;
```